

Cahier de conception

Plateforme Comptabilité — ERP Yowyob

Auteur : AZANGUE LEONEL DELMAT
Superviseur : Pr Dr Eng Djotio Thomas NDIE

27 avril 2026

Table des matières

1	Introduction	5
1.1	Objectif	5
1.2	Périmètre	5
2	Vue d'ensemble du système	6
2.1	Stack technique	6
2.2	Organisation du code	6
2.3	Architecture logique (couches)	7
3	Exigences (extraites du code et du README)	8
3.1	Exigences fonctionnelles	8
3.2	Exigences non fonctionnelles	8
4	Multi-tenant et isolation des données	9
4.1	Principe	9
4.2	Résolution du tenant	9
4.2.1	Règles de résolution	9
5	Sécurité	10
5.1	Authentification	10
5.2	Autorisation	10
5.3	Gestion des erreurs	10
6	Conception des données	11
6.1	Stratégie de migration	11
6.2	Entités principales (comptabilité)	11
6.3	Modèle relationnel (extrait)	11
7	API REST (reverse engineering) — Comptabilité	13
7.1	Conventions	13
7.2	Exemple : <code>/api/accounting/ecritures</code>	13
7.3	Couverture des contrôleurs du module comptable	13
7.4	Annexe API exhaustive	14
8	Modélisation mathématique (à ne pas négliger)	15
8.1	Notations	15
8.2	Invariant de la double entrée	15
8.3	Balance, grand livre, compte de résultat, bilan (formalisme)	15
8.3.1	Grand livre (par compte)	15

8.3.2	Balance des comptes	16
8.3.3	Compte de résultat	16
8.3.4	Bilan	16
8.4	Contraintes de période	16
8.5	Machine à états (écritures)	17
8.6	Lettrage	17
8.7	Pointage bancaire	17
9	Conception des services (extrait)	18
9.1	Service des écritures	18
10	Déploiement et exploitation	19
10.1	Configuration	19
10.2	Docker / Compose	19
10.3	CI/CD	19
11	Risques, dette technique et recommandations	20
A	Annexes	21
B	Annexe A — Spécification API (exhaustive)	22
B.1	Introduction	22
B.2	EcritureComptableController — /api/accounting/ecritures	22
B.3	CompteController — /api/accounting/comptes	22
B.4	PlanComptableController — /api/accounting/plan-comptable	23
B.5	JournalComptableController — /api/accounting/journals	23
B.6	PeriodeComptableController — /api/accounting/periodes	23
B.7	ExerciceComptableController — /api/accounting/exercices	24
B.8	RapportController — /api/accounting/rapport	24
B.9	PointageController — /api/accounting/pointage	24
B.10	LettrageController — /api/comptable/lettrage	25
B.11	Note	25
C	Annexe B — Dictionnaire de données (exhaustif)	26
C.1	Principe	26
C.2	Tables cœur (comptabilité)	26
C.2.1	organizations	26
C.2.2	comptes	26
C.2.3	plans_comptables	26
C.2.4	journals_comptables	26
C.2.5	exercices_comptables	27
C.2.6	periodes_comptables	27
C.2.7	ecritures_comptables	27
C.2.8	details_ecritures	27
C.2.9	journal_audit	27
C.3	Tables avancées (extraits)	28
C.3.1	brouillards_comptables	28
C.3.2	immobilisations / amortissements_lignes	28
C.4	Note de complétude	28

C.5 Principaux fichiers de référence	28
--	----

Table des figures

2.1	Architecture en couches (vue reverse-engineered)	7
-----	--	---

Chapitre 1

Introduction

1.1 Objectif

Ce document est un **cahier de conception complet** de la **plateforme comptabilité** de l'ERP Yowyob. Il vise à décrire l'architecture, les modèles de données, les flux applicatifs, l'API, la sécurité et le multi-tenant, ainsi qu'une **modélisation mathématique** des invariants comptables (double entrée, contraintes de périodes, lettrage, pointage).

1.2 Périmètre

Le périmètre porte principalement sur le module **accounting** (comptabilité générale OHADA) et les composants transverses **common** et **config**.

Chapitre 2

Vue d'ensemble du système

2.1 Stack technique

Composant	Choix technologique
Langage / Runtime	Java 21
Framework HTTP	Spring Boot 3.5.3, Spring WebFlux (réactif)
Base de données	PostgreSQL, accès réactif via R2DBC
Gestion de schéma	Liquibase (exécuté au démarrage)
Sécurité	Spring Security WebFlux + JWT (validation via service externe)
Composants optionnels	Redis (cache), Kafka (événements/audit), Elasticsearch (recherche)

TABLE 2.1 – Stack technique de la plateforme comptabilité

2.2 Organisation du code

Le code est organisé en trois ensembles cohérents :

1. `com.yowyob.erp.accounting` : domaine comptable (API, services, persistance, entités, DTO).
2. `com.yowyob.erp.common` : éléments transverses (wrappers de réponse, exceptions, objets communs).
3. `com.yowyob.erp.config` : configuration technique (auth/JWT, CORS, multi-organisation, R2DBC, Redis, Kafka, Swagger).

2.3 Architecture logique (couches)

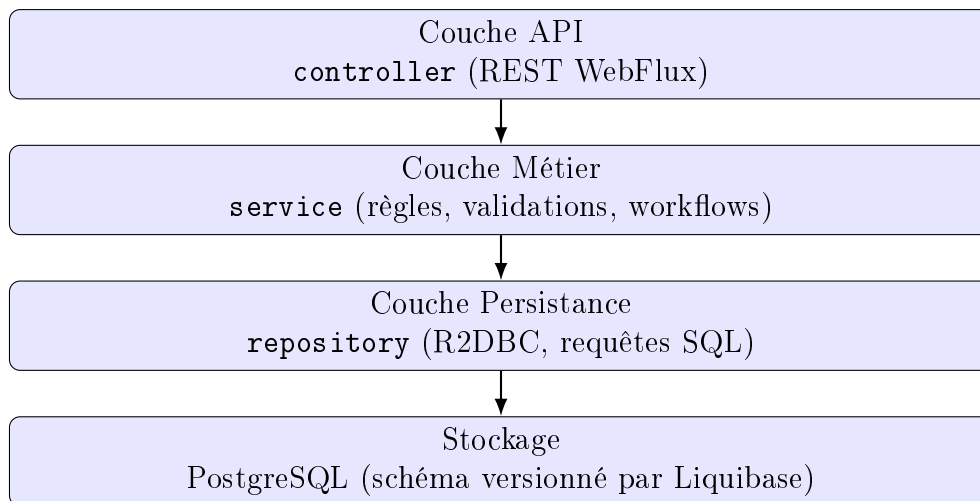


FIGURE 2.1 – Architecture en couches (vue reverse-engineered)

Chapitre 3

Exigences (extraites du code et du README)

3.1 Exigences fonctionnelles

- RF-1.** Gestion du plan comptable OHADA, des comptes, journaux, écritures et détails d'écritures.
- RF-2.** Gestion des exercices et périodes comptables, avec mécanismes de clôture.
- RF-3.** Lettrage et pointage des lignes d'écritures.
- RF-4.** Journalisation (audit) des actions et traçabilité des changements.
- RF-5.** Génération d'écritures à partir d'objets métier comptabilisables (`ComptableObject`).

3.2 Exigences non fonctionnelles

- RNF-1.** Réactivité et scalabilité (WebFlux + R2DBC) pour supporter des volumes importants d'écritures.
- RNF-2.** Isolation multi-organisation : toutes les données métier sont partitionnées par `organization_id`.
- RNF-3.** Traçabilité : journal d'audit et champs d'audit (`created_at`, `updated_at`, `created_by`, `updated_by`).
- RNF-4.** Sécurité : authentification JWT, contrôle d'accès par rôles, CORS configurable.

Chapitre 4

Multi-tenant et isolation des données

4.1 Principe

L'isolation des données est réalisée par propagation d'un identifiant d'organisation (`organizationId`) dans le **Reactor Context**. Chaque requête API est associée à une organisation déterminée via un header configurable (par défaut **X-Tenant-ID**) ou un paramètre de requête `tenantId` (cas SSE).

4.2 Résolution du tenant

Un `WebFilter` (`OrganizationWebFilter`) extrait l'identifiant et l'injecte dans le contexte réactif.

4.2.1 Règles de résolution

1. Lire le header **X-Tenant-ID** (ou nom configuré).
2. Sinon, lire le query param `tenantId`.
3. Sinon, utiliser une valeur par défaut (`app.organization.default-organization`).

Chapitre 5

Sécurité

5.1 Authentification

L'authentification repose sur un token `Bearer` dans `Authorization`. Un filtre (`JwtAuthenticationFilter`) appelle un service `AuthService` pour valider le token via une API externe (`auth.api.url`). En cas de succès, il place une authentification avec des autorités `ROLE_X` dans le contexte de sécurité réactif.

5.2 Autorisation

La chaîne de sécurité (`SecurityWebFilterChain`) définit des règles `pathMatchers`. **Point critique** : une règle de test autorise `/api/accounting/**` sans token (mentionnée comme `TEMP` dans le code).

5.3 Gestion des erreurs

Les exceptions sont normalisées via `GlobalExceptionHandler` et retournées sous une forme `ApiResponseWrapper`.

Chapitre 6

Conception des données

6.1 Stratégie de migration

Le schéma SQL est versionné par Liquibase et orchestré depuis `db.changelog-master.xml`. Une migration importante (`changeset-017-unify-tenant-org.xml`) unifie les notions `tenant` et `organization` et renomme les colonnes `tenant_id` en `organization_id`.

6.2 Entités principales (comptabilité)

Ce projet utilise des entités R2DBC annotées `@Table`. Exemples représentatifs :

`EcritureComptable` → `ecritures_comptables`

`DetailEcriture` → `details_ecritures`

`Compte` → `comptes`

`JournalComptable` → `journaux_comptables`

`PeriodeComptable` → `periodes_comptables`

`JournalAudit` → `journal_audit`

6.3 Modèle relationnel (extrait)

Table	Rôle
<code>organizations</code>	Organisation (tenant). Identité et métadonnées (nom, email, téléphone, <code>tax_id</code> , etc.).
<code>agences</code>	Agences rattachées à une organisation.
<code>comptes</code>	Comptes comptables (classe OHADA, type, solde).
<code>journaux_comptables</code>	Journaux (achats, ventes, trésorerie, etc.).
<code>exercices_comptables</code>	Exercice fiscal/comptable (dates, clôture).
<code>periodes_comptables</code>	Périodes (liées à un exercice), ouverture/clôture.
<code>ecritures_comptables</code>	Entête d'écriture comptable (numéro, libellé, totaux, statut, validée).
<code>details_ecritures</code>	Lignes de débit/crédit par compte (lettrage, pointage).

Table	Rôle
journal_audit	Journalisation des actions (création, validation, suppression...).

Chapitre 7

API REST (reverse engineering) — Comptabilité

7.1 Conventions

Les endpoints sont regroupés sous le préfixe `/api/accounting/` (ex : `/ecritures`, `/comptes`, `/journaux`, etc.). Les contrôleurs sont annotés `@RestController` et retournent des `Mono<ResponseEntity<ApiResponseWrapper<...>>>`.

7.2 Exemple : `/api/accounting/ecritures`

Le contrôleur `EcritureComptableController` expose notamment :

`POST /api/accounting/ecritures` — création

`PUT /api/accounting/ecritures/{id}` — mise à jour

`POST /api/accounting/ecritures/{id}/validate` — validation

`GET /api/accounting/ecritures` — liste

`GET /api/accounting/ecritures/non-validated` — non validées

`GET /api/accounting/ecritures/search` — recherche

`POST /api/accounting/ecritures/generate` — génération depuis `ComptableObject`

`PATCH /api/accounting/ecritures/{id}/cancel` — annulation

`PATCH /api/accounting/ecritures/{id}/deactivate` — désactivation

`DELETE /api/accounting/ecritures/{id}` — suppression

7.3 Couverture des contrôleurs du module comptable

Le module comptabilité expose (au moment du reverse engineering) les contrôleurs suivants :

`CompteController` → `/api/accounting/comptes`

`PlanComptableController` → `/api/accounting/plan-comptable`

`JournalComptableController` → `/api/accounting/journals`

`EcritureComptableController` → `/api/accounting/ecritures`

PeriodeComptableController → /api/accounting/periodes

ExerciceComptableController → /api/accounting/exercices

RapportController → /api/accounting/rapport

PointageController → /api/accounting/pointage

LettrageController → /api/comptable/lettrage (préfixe atypique)

Autres contrôleurs du domaine (taxes, devises, immobilisations, budgets, paie, brouillards, etc.)

7.4 Annexe API exhaustive

La liste exhaustive des endpoints (tous contrôleurs, méthodes, chemins, verbes HTTP) est fournie en annexe.

Chapitre 8

Modélisation mathématique (à ne pas négliger)

8.1 Notations

Soit une organisation $o \in \mathcal{O}$. Une écriture comptable e est un couple :

$$e = (H, L)$$

où H est l'entête (journal, période, date, statut) et L est l'ensemble des lignes $l \in L$.

Chaque ligne l est associée à un compte $a \in \mathcal{A}$ et porte des montants (d_l, c_l) (débit, crédit) avec $d_l \geq 0, c_l \geq 0$.

8.2 Invariant de la double entrée

Une écriture est **équilibrée** si :

$$\sum_{l \in L} d_l = \sum_{l \in L} c_l$$

Dans l'implémentation, un seuil de tolérance $\varepsilon = 0,01$ est utilisé :

$$\left| \sum d_l - \sum c_l \right| \leq \varepsilon$$

Cette contrainte est vérifiée lors de la création/mise à jour/validation (`validateBalance`).

8.3 Balance, grand livre, compte de résultat, bilan (formalisme)

8.3.1 Grand livre (par compte)

Soit un compte a et une période $[t_0, t_1]$. On définit l'ensemble des lignes :

$$L(a, t_0, t_1) = \{ l \mid \text{account}(l) = a \wedge t_0 \leq \text{date}(l) \leq t_1 \}$$

Les totaux de mouvements sont :

$$D(a) = \sum_{l \in L(a, t_0, t_1)} d_l, \quad C(a) = \sum_{l \in L(a, t_0, t_1)} c_l$$

Le solde de clôture s'exprime à partir d'un solde d'ouverture $S_0(a)$:

$$S_1(a) = S_0(a) + D(a) - C(a)$$

(la convention de signe peut varier selon la nature du compte ; le système peut aussi porter des soldes débit/crédit séparés).

8.3.2 Balance des comptes

La balance est une agrégation sur un ensemble de comptes $\mathcal{A}$ produisant, pour chaque a , un quadruplet :

$$(S_0(a), D(a), C(a), S_1(a))$$

La contrainte de cohérence globale (si tout est saisi en double entrée) est :

$$\sum_{a \in \mathcal{A}} D(a) = \sum_{a \in \mathcal{A}} C(a)$$

8.3.3 Compte de résultat

Soit $\mathcal{A}_{\text{prod}}$ l'ensemble des comptes de produits (classe 7, selon paramétrage), et $\mathcal{A}_{\text{ch}}$ l'ensemble des comptes de charges (classe 6). On définit :

$$\text{PRODUITS} = \sum_{a \in \mathcal{A}_{\text{prod}}} (C(a) - D(a)), \quad \text{CHARGES} = \sum_{a \in \mathcal{A}_{\text{ch}}} (D(a) - C(a))$$

Le résultat net sur la période est :

$$\text{RN} = \text{PRODUITS} - \text{CHARGES}$$

8.3.4 Bilan

Soit $\mathcal{A}_{\text{actif}}$ et $\mathcal{A}_{\text{passif}}$ (classes 1–5 selon mapping OHADA et paramétrage). On agrège les soldes :

$$\text{ACTIF} = \sum_{a \in \mathcal{A}_{\text{actif}}} \max(S_1(a), 0), \quad \text{PASSIF} = \sum_{a \in \mathcal{A}_{\text{passif}}} \max(-S_1(a), 0)$$

et une propriété attendue est l'équilibre bilan :

$$\text{ACTIF} = \text{PASSIF}$$

La mise en œuvre concrète dépend des conventions de signe et de la classification des comptes.

8.4 Contraintes de période

Soit une période p avec un indicateur $\text{closed}(p) \in \{0, 1\}$. La règle métier est :

$$\text{closed}(p) = 1 \Rightarrow \neg \text{create}(e) \wedge \neg \text{update}(e)$$

c'est-à-dire : aucune écriture ne peut être créée/modifiée dans une période clôturée.

8.5 Machine à états (écritures)

Soit $S = \{\text{BROUILLON}, \text{VALIDE}, \text{ANNULE}\}$. Transitions autorisées (extrait des services) :

$$\text{BROUILLON} \xrightarrow{\text{validate}} \text{VALIDE}, \quad \text{BROUILLON} \xrightarrow{\text{cancel}} \text{ANNULE}, \quad \text{VALIDE} \xrightarrow{\text{cancel}} \text{ANNULE}$$

Avec contrainte : validate n'est possible que si l'écriture est équilibrée.

8.6 Lettrage

Chaque ligne peut être lettrée (**lettree**). Une formalisation possible consiste à définir une relation de lettrage $\mathcal{R}$ sur les lignes d'un même compte :

$$\mathcal{R} \subseteq L \times L$$

et imposer qu'un groupe lettré $G \subseteq L$ respecte l'équilibre partiel :

$$\sum_{l \in G} d_l = \sum_{l \in G} c_l$$

Le code stocke actuellement l'état de lettrage au niveau des lignes (**lettree**, **date_lettrage**) ; les règles de groupement peuvent être portées par un service de lettrage.

8.7 Pointage bancaire

Pour une ligne l , le pointage est $pointee(l) \in \{0, 1\}$ avec une référence bancaire $\text{ref}(l)$. Une contrainte d'unicité métier recommandée est :

$$\forall l_1 \neq l_2, \text{ref}(l_1) = \text{ref}(l_2) \wedge pointee(l_1) = pointee(l_2) = 1 \Rightarrow \text{invalide}$$

(à implémenter éventuellement via contrainte BD ou règle applicative selon le besoin).

Chapitre 9

Conception des services (extrait)

9.1 Service des écritures

Le service `EcritureComptableService` illustre les règles métier :

- création d'une écriture → génération d'un numéro `ECR-xxxxxxx`
- contrôle période non clôturée
- validation d'équilibre débit/crédit
- journalisation (table audit) + émission d'événement
- cache Redis (liste, recherche) avec invalidation lors des mutations

Chapitre 10

Déploiement et exploitation

10.1 Configuration

La configuration est centralisée dans `application.properties` et surchargée par variables d'environnement :

DB : `DB_HOST`, `DB_PORT`, `DB_NAME`, `POSTGRES_USER`, `POSTGRES_PASSWORD`

Auth : `AUTH_API_URL`, `JWT_SECRET`

Features : `SPRING_KAFKA_ENABLED`, `SPRING_ELASTICSEARCH_ENABLED`, `SPRING_CACHE_TYPE`

10.2 Docker / Compose

`docker-compose.yml` démarre Postgres, Redis, Kafka, Elasticsearch et l'application.

`Dockerfile` compile et exécute l'application.

10.3 CI/CD

Le dépôt contient un workflow GitHub Actions pour construire et publier une image Docker.

Chapitre 11

Risques, dette technique et recommandations

Sécurité : routes comptables temporairement `permitAll` → à corriger avant production.

Secrets : éviter tout secret versionné (fichiers `.env`, mots de passe compose).

Cohérence tenant : s'assurer que toutes les requêtes filtrent bien par `organization_id`.

Transactions réactives : clarifier les transaction managers (R2DBC vs JDBC/Liquibase).

Observabilité : compléter `Actuator/metrics` et corrélation `requestId/organizationId`.

Annexe A

Annexes

Annexe B

Annexe A — Spécification API (exhaustive)

B.1 Introduction

Cette annexe recense les endpoints **du module Comptabilité** par contrôleur, tels qu'observés dans le code (`src/main/java/com/yowyob/erp/accounting/controller`).

B.2 `EcritureComptableController` — `/api/accounting/ecritures`

POST `/api/accounting/ecritures` — créer une écriture (`EcritureComptableDto`)

PUT `/api/accounting/ecritures/{id}` — modifier une écriture (`EcritureComptableDto`)

POST `/api/accounting/ecritures/{id}/validate` — valider une écriture

GET `/api/accounting/ecritures` — lister

GET `/api/accounting/ecritures/{id}` — obtenir par id

GET `/api/accounting/ecritures/non-validated` — lister non validées

GET `/api/accounting/ecritures/search` — recherche (start/end/journalId)

POST `/api/accounting/ecritures/generate` — générer depuis `ComptableObject`

DELETE `/api/accounting/ecritures/{id}` — supprimer

PATCH `/api/accounting/ecritures/{id}/cancel` — annuler

PATCH `/api/accounting/ecritures/{id}/deactivate` — désactiver (soft delete)

B.3 `CompteController` — `/api/accounting/comptes`

POST `/api/accounting/comptes` — créer un compte (`CompteDto`)

POST `/api/accounting/comptes/generate` — générer un compte (tiers etc.)

GET `/api/accounting/comptes` — lister

GET `/api/accounting/comptes/{id}` — obtenir par id

GET `/api/accounting/comptes/search?no_compte=...` — recherche

GET `/api/accounting/comptes/clients` — comptes clients

GET /api/accounting/comptes/suppliers — comptes fournisseurs
GET /api/accounting/comptes/banks — comptes bancaires
GET /api/accounting/comptes/cash — comptes caisse
GET /api/accounting/comptes/type/{type} — comptes par type
PUT /api/accounting/comptes/{id} — mise à jour
DELETE /api/accounting/comptes/{id} — suppression

B.4 PlanComptableController — /api/accounting/plan-comptable

POST /api/accounting/plan-comptable/admin/organizations/{organizationId}/plan-comptable — initialisation OHADA 2025
POST /api/accounting/plan-comptable — créer un compte de plan (PlanComptableDto)
GET /api/accounting/plan-comptable/{id} — obtenir par id
GET /api/accounting/plan-comptable — lister
GET /api/accounting/plan-comptable/actifs — lister actifs
GET /api/accounting/plan-comptable/prefix/{prefix} — filtrer par préfixe
GET /api/accounting/plan-comptable/classe/{classe} — filtrer par classe OHADA
PUT /api/accounting/plan-comptable/{id} — mise à jour
POST /api/accounting/plan-comptable/import — import (CSV/XLSX multipart)
DELETE /api/accounting/plan-comptable/{id} — désactivation (soft delete)

B.5 JournalComptableController — /api/accounting/journals

POST /api/accounting/journals — créer (JournalComptableDto)
PUT /api/accounting/journals/{id} — mise à jour
GET /api/accounting/journals/{id} — obtenir
GET /api/accounting/journals/{id}/comptes — comptes distincts utilisés
GET /api/accounting/journals — lister
GET /api/accounting/journals/active — lister actifs
DELETE /api/accounting/journals/{id} — supprimer

B.6 PeriodeComptableController — /api/accounting/periodes

POST /api/accounting/periodes — créer (PeriodeComptableDto)
GET /api/accounting/periodes/{id} — obtenir
GET /api/accounting/periodes — lister
GET /api/accounting/periodes/code/{code} — obtenir par code
GET /api/accounting/periodes/by-date?date=... — obtenir par date
GET /api/accounting/periodes/non-closed — lister ouvertes

GET /api/accounting/periodes/range?start_date=...&end_date=... — intervalle
PUT /api/accounting/periodes/{id} — mise à jour
PUT /api/accounting/periodes/{id}/close — clôture
DELETE /api/accounting/periodes/{id} — suppression

B.7 ExerciceComptableController — /api/accounting/exercices

POST /api/accounting/exercices — créer (ExerciceComptableDto)
GET /api/accounting/exercices/{id} — obtenir
GET /api/accounting/exercices/{id}/periodes — périodes de l'exercice
GET /api/accounting/exercices — lister
GET /api/accounting/exercices/active?date=... — actif pour date
PUT /api/accounting/exercices/{id} — mise à jour
POST /api/accounting/exercices/{id}/close — clôture exercice
DELETE /api/accounting/exercices/{id} — suppression
PATCH /api/accounting/exercices/{id}/deactivate — désactivation

B.8 RapportController — /api/accounting/rapport

GET /api/accounting/rapport/bilan — bilan (JSON)
GET /api/accounting/rapport/compte-resultat — compte de résultat (JSON)
GET /api/accounting/rapport/flux-tresorerie — flux de trésorerie (JSON)
GET /api/accounting/rapport/resume-executif — résumé exécutif (JSON)
GET /api/accounting/rapport/grand-livre — grand livre (JSON)
GET /api/accounting/rapport/balance — balance (JSON)
GET /api/accounting/rapport/balance-agee — balance âgée (JSON)
GET /api/accounting/rapport/bilan/export/pdf — bilan (PDF)
GET /api/accounting/rapport/compte-resultat/export/pdf — compte résultat (PDF)
GET /api/accounting/rapport/grand-livre/export/pdf — grand livre (PDF)
GET /api/accounting/rapport/balance/export/pdf — balance (PDF)
GET /api/accounting/rapport/balance-agee/export/pdf — balance âgée (PDF)

B.9 PointageController — /api/accounting/pointage

POST /api/accounting/pointage/import — import relevé bancaire (CSV) + pointage

B.10 LettrageController — /api/comptable/lettrage

POST /api/comptable/lettrage/auto — lettrage automatique

GET /api/comptable/lettrage/status — statut (placeholder)

B.11 Note

Cette annexe sera étendue au même format pour les autres contrôleurs du domaine (taxes, devises, immobilisations, budgets, paie, brouillards, regularisations, rapprochement, relevés, notifications, attachments, etc.).

Annexe C

Annexe B — Dictionnaire de données (exhaustif)

C.1 Principe

Le schéma est géré par **Liquibase**. Une migration structurante (`changeset-017-unify-tenant-org`) renomme `tenant_id` en `organization_id` sur la majorité des tables et unifie `tenants` et `organizations`.

C.2 Tables cœur (comptabilité)

C.2.1 organizations

Rôle : organisation (tenant) — unité d'isolation des données.

Identifiant : `id` (UUID, PK)

Attributs (extrait) : `name`, `email`, `telephone`, `address`, `tax_id`, timestamps.

C.2.2 comptes

PK : `id` (UUID)

FK : `organization_id` → `organizations(id)` (après unification)

Champs : `no_compte`, `libelle`, `classe`, `type_compte`, `solde`, `actif`, `audit`.

Unicité : (`organization_id`, `no_compte`) (contrainte unique).

C.2.3 plans_comptables

PK : `id` (UUID)

FK : `organization_id` → `organizations(id)`

Champs : `classe`, `no_compte`, `libelle`, `notes`, `actif`, `audit`.

C.2.4 journaux_comptables

PK : `id` (UUID)

FK : `organization_id` → `organizations(id)`

Champs : code_journal, libelle, type_journal, actif, audit.

C.2.5 exercices_comptables

PK : id (UUID)

FK : organization_id → organizations(id)

Champs : code, libelle, date_debut, date_fin, cloture, audit.

C.2.6 periodes_comptables

PK : id (UUID)

FK : organization_id → organizations(id)

FK : exercice_id → exercices_comptables(id)

Champs : code, date_debut, date_fin, cloturee, date_cloture, audit.

C.2.7 ecritures_comptables

PK : id (UUID)

FK : organization_id → organizations(id)

FK : journal_id → journaux_comptables(id)

FK : periode_id → periodes_comptables(id)

Champs : numero_ecriture (unique), libelle, date_ecriture, totaux débit/crédit, validee, statut, actif, audit.

C.2.8 details_ecritures

PK : id (UUID)

FK : organization_id → organizations(id)

FK : ecriture_id → ecritures_comptables(id)

FK : compte_id → plans_comptables(id) (selon changeset initial)

Champs : sens, montant_debit, montant_credit, lettree, date_lettrage, pointee, reference_bancaire, audit.

C.2.9 journal_audit

PK : journal_audit_id (UUID) / entité JournalAudit

FK : organization_id → organizations(id)

FK : ecriture_id → ecritures_comptables(id) (SET NULL)

Champs : action, date_action, utilisateur, details, avant/après, audit.

C.3 Tables avancées (extraits)

C.3.1 brouillards_comptables

Table de brouillard/draft (source externe, pièce, JSONB `data_json`, statut, lien optionnel vers `ecriture_id`).

C.3.2 immobilisations / amortissements_lignes

Gestion des immobilisations et amortissements, avec liens vers des comptes (`compte_immo_id`, `compte_amort_id`, `compte_dotation_id`) et exercice.

C.4 Note de complétude

Ce dictionnaire est aligné sur les changesets observés (001..020). Il peut être enrichi section par section avec :

- types exacts, nullabilité et valeurs par défaut,
- index/contraintes (unique, FK, cascade),
- règles métier associées par service.

C.5 Principaux fichiers de référence

```
pom.xml
src/main/resources/application.properties
src/main/resources/db/changelog/db.changelog-master.xml
src/main/java/com/yowyob/erp/config/auth/SecurityConfig.java
src/main/java/com/yowyob/erp/config/organization/OrganizationWebFilter.java
```